读懂 RustSBI-QEMU 的 `main.rs`

这份 `main.rs` 是一个运行在 RISC-V M 模式的固件入口文件。它把平台驱动、SBI 服务和陷入处理接起来，再把控制权交给运行在 S 模式的内核。全文按你贴出的 393 行源码编号。

附件行号	读什么	对应页
1–61	编译属性、模块和 `_start`	01
64–156	`rust_main` 的全局初始化和每 hart 初始化	02
158–178、196–220	PMP 如何隔离固件，`boot` 如何准备 S 模式上下文	03
180–189、221–309	`fast_handler` 如何分发 SBI 调用和异常	04
311–393	参数结构、串口、SBI 后端和 HSM 实现	05

第 1–40 行 · 为什么需要自定义入口

`#![no_std]` 关闭隐式引入的标准库，裸机代码仍可以使用 core。`#![no_main]` 关闭常规 Rust 入口，由链接后的 `_start` 接管。`#![feature(...)]` 是这份历史源码使用的特性开关，`#![deny(warnings)]` 将编译警告视为错误。

`device_tree` 读平台信息，`uart16550`、`clint` 和 `qemu_test` 操作设备。`trap_stack`、`trap_vec` 和 `riscv_spec` 处理陷入栈、入口和寄存器。`constants` 规定 S 模式入口 `0x80200000`、每 hart 16 KiB 的栈和此实现支持的 hart 数量上限。

第 47–61 行 · `_start` 的三步

```
#[naked]
#[no_mangle]
#[link_section = ".text.entry"]
unsafe extern "C" fn _start() -> ! {
    asm!(
        "   call {locate_stack}
            call {rust_main}
            j    {trap}
        ",
        locate_stack = sym trap_stack::locate,
        rust_main    = sym rust_main,
        trap         = sym trap_vec,
        options(noreturn),
    )
}
```

`#[naked]` 让汇编负责入口处的栈设置，`#[no_mangle]` 保留符号名，`#[link_section]` 把入口放到指定段。先定位当前 hart 的栈，再调用固件的 Rust 函数，最后跳到 `trap_vec`。`sym` 把 Rust 项的符号地址交给汇编，`options(noreturn)` 表示控制流会从这里跳走。

`rust_main` 怎样初始化平台

附件第 64–156 行

固件入口接收两个参数。`hartid` 是当前硬件线程编号。首次启动的 `opaque` 来自 QEMU，里面是设备树地址，因此能传给 `device_tree::parse(opaque)`。

先分出首个 hart 和其余 hart

```
extern "C" fn rust_main(hartid: usize, opaque: usize) {
    static GENESIS: AtomicBool = AtomicBool::new(true);
    static BOARD_INFO: Once<BoardInfo> = Once::new();

    // 全局初始化过程
    if GENESIS.swap(false, Ordering::AcqRel) {
    ...
```

`GENESIS.swap(false, Ordering::AcqRel)` 原子地写入 `false` 并返回旧值。只有第一个执行交换的 hart 看到 `true`，进入全局初始化。`sbss`、`ebss` 是链接符号，循环以 8 字节为单位对 `[sbss, ebss)` 写零。后续的 `BOARD_INFO.call_once` 发布设备树解析结果。

行号	这几段代码做什么
82–90	解析 DTB，得到 UART、CLINT、QEMU Test 设备和内存范围。初始化设备和控制台。`option_env!("LOG")` 读取编译这个固件时的环境变量。`dbcn::init` 用内核入口到 RAM 末尾的范围配置调试控制台后端。
91–113	打印启动横幅。版本取自库和包元数据，平台名、hart 数量、内存和 DTB 范围取自 `board_info`，固件地址取自 `_start` 的地址。
114–132	把平台后端装进 `SBI`，设置并打印 PMP，准备本 hart 的陷入栈，然后向本 hart 的 HSM 登记内核入口和 `opaque`。
133–138	其他 hart 通过 `BOARD_INFO.wait()` 等待设备信息，再各自设置 PMP 和陷入栈。
139–150	清理本地中断相关状态，配置中断和异常委托、计数器访问以及 `mtvec`。清掉 S 模式 `ecall` 的委托位，保证 SBI 请求进入 M 模式。

> `local_remote_hsm().start(...)` 在这里登记启动请求。`rust_main` 返回后，`_start` 跳到 `trap_vec`，由 `fast_handler` 取出请求并建立内核的启动上下文。

PMP 和内核启动上下文

附件第 158–178、196–220 行

`set_pmp` 如何分出可访问的物理区间

这段使用 TOR（Top of Range）。一项的上界来自当前 `pmpaddr`，下界来自前一项。`mem.start >> 2` 和其他右移操作把字节地址转换成 PMP 地址寄存器使用的单位。

项	区间	S 模式权限
0	`OFF`，设定后续区间的起始边界	本项关闭
1	低地址到 RAM 起点，包含平台外设区	读、写
2	RAM 起点到 `SUPERVISOR_ENTRY`，包含固件	禁止访问
3	`SUPERVISOR_ENTRY` 到 RAM 末尾	读、写、执行
4	RAM 末尾之后，由末项 PMP 地址确定上界	读、写

`fast_handler` 收到启动请求后

```
#[inline]
fn boot(mut ctx: FastContext, start_addr: usize, opaque: usize) -> FastResult {
    unsafe {
        sstatus::clear_sie();
        satp::write(0);
    }
    ctx.regs().a[0] = hart_id();
    ctx.regs().a[1] = opaque;
    ctx.regs().pc = start_addr;
    ctx.call(2)
}
loop {
    match local_hsm().start() {
        Ok(supervisor) => {
            mstatus::update(|bits| {
                *bits &= !mstatus::MPP;
                *bits |= mstatus::MPIE | mstatus::MPP_SUPERVISOR;
            });
            mie::write(mie::MSIE | mie::MTIE);
            break boot(ctx, supervisor.start_addr, supervisor.opaque);
```

`local_hsm().start()` 取到 `Supervisor` 时，处理函数先把 `mstatus.MPP` 设为 S 模式，把 `MPIE` 置位，并打开 M 模式的软件和计时器中断位。随后 `boot` 清除 S 模式的全局中断使能，写入 `satp = 0`，让内核从裸地址转换模式开始。

`boot` 给新上下文填好 `a0 = hart ID`、`a1 = opaque` 和目标 PC。`ctx.call(2)` 表示按两个参数的快速调用路径交给底层恢复代码。底层执行 `mret` 时，CPU 才根据 `mepc` 和 `mstatus.MPP` 进入 S 模式内核。

fast-trap 上下文模型 · RISC-V 特权架构

`fast_handler` 怎样处理 SBI 调用

附件第 180–189、206–309 行

入口用 `FastContext` 访问保存的现场。原来的 `a0` 通过 `ctx.a0()` 读取，`a1..a7` 作为其余参数传入。循环每次先检查 HSM，然后再决定如何处理 `mcause`。

状态	执行路径
有启动请求	调用上一页的 `boot`，进入新上下文。
`HART_STOP`	只保留机器软件中断使能，执行 `wfi` 等待唤醒，清除软件中断挂起后再次检查。
其他状态	读取 `mcause`。S 模式 `ecall` 交给 SBI 分发，未支持的陷入进入诊断和 panic 路径。

从 `ecall` 取参数并写回结果

```
T::Exception(E::SupervisorEnvCall) => {
    use sbi_spec::{base, hsm, legacy};
    let mut ret = unsafe { SBI.assume_init_mut() }.handle_ecall(
        a7,
        a6,
        [ctx.a0(), a1, a2, a3, a4, a5],
    );
```

对标准 SBI 调用，`a7` 是扩展 ID，`a6` 是函数 ID，`a0..a5` 是最多六个实参。这里通过 `SBI.assume_init_mut()` 取得已注册的后端并调用 `handle_ecall`。固件把返回码和返回值放回 `a0`、`a1`，其余原始参数保留。`mepc::next()` 推进返回 PC，避免返回后再次执行同一条 `ecall`。

```
ctx.regs().a = [ret.error, ret.value, a2, a3, a4, a5, a6, a7];
mepc::next();
break ctx.restore();
```

为什么中间还有两层匹配

分发成功时，hart 停止请求会回到处理循环。源码还为成功的非保留挂起预留了按恢复地址重新启动的分支。查询旧控制台扩展时，代码补上兼容返回值。

分发失败时，代码按 `a7` 检查旧版 `putchar` 和 `getchar`，分别直接打印或轮询 UART。`getchar` 在这份实现里会等待收到一个字节。

其他陷入会打印原因、`mstatus`、`mepc` 和 `mtval`。第 299–309 行的 panic handler 再打印 hart 和 panic 信息，并请求 QEMU 以系统故障原因关机。

SBI 调用与返回约定

文件末尾的三个后端

附件第 311–393 行

`Supervisor` 和 `Console`

`Supervisor` 保存一次启动所需的入口地址和透传参数。首次启动时透传参数是 DTB 地址，其他 hart 的启动请求可携带调用者提供的值。`Console` 实现 `rcore_console::Console`。`put_char` 持有 UART 锁并重试直到写入一个字节。`put_str` 处理部分写入，每次从剩余字节切片继续写。

`FixedRustSBI` 把设备接入 SBI

```
static mut SBI: MaybeUninit<FixedRustSBI> = MaybeUninit::uninit();

#[derive(RustSBI)]
struct FixedRustSBI<'a> {
    #[rustsbi(ipi, timer)]
    clint: &'a clint::Clint,
    hsm: Hsm,
    reset: &'a qemu_test::QemuTest,
    dbcn: &'a dbcn::DBCN,
}
```

`MaybeUninit` 让全局对象等到设备初始化后再填入。第 116 行写入 `FixedRustSBI`，第 225 行在处理 SBI 调用时读取它。`#[derive(RustSBI)]` 生成分发逻辑，`#[rustsbi(ipi, timer)]` 指明同一个 CLINT 实现同时提供 IPI 和计时器服务。

`Hsm` 管理每个 hart 的运行状态

方法	这个实现的行为
`hart_start`	查找目标 hart。登记 `start_addr` 和 `opaque` 后用 `clint::set_msip(hartid)` 唤醒它。根据结果返回成功、已启动或参数无效。
`hart_stop`	将当前 hart 标为停止。处理函数回到循环后会走 `HART_STOP` 路径。
`hart_get_status`	按 hart ID 查询状态并编码为 SBI 返回值。找不到时返回参数无效。
`hart_suspend`	保留上下文的挂起会标记挂起、执行 `wfi`、唤醒后恢复状态。这个后端对其他挂起类型返回 `not_supported()`。

> **对照两个分支** · 第 235–238 行只有在非保留挂起请求**成功**时才调用 `boot`。第 382–390 行显示，本文件的 `Hsm` 后端只接受保留挂起，因此当前实现的非保留请求不会走到这个成功分支。

主资料为你粘贴的 `main.rs`。页内行号按该附件计数。上游 RustSBI-QEMU main.rs